

Fase 0 — bots, credenciais e grupo de admin · status de entrega

Por que este arquivo não tem número de bloco. A trilha da automação é a lista "OCR Contratos (WhatsApp) — Breno" do ClickUp, que tem numeração própria de 1 a 8 e não usa blocos `m.n`. Os blocos `m.n` são da lista "Plataforma Gestão de Obras", outra trilha — usar um número de lá aqui seria o que o `CLAUDE.md` proíbe. Nome descritivo, então, e a pasta é `docs/automacao/`. Mesma regra da Fase 1.

Data: 2026-09-22 · Branch: `feature-dev-breno-automacao` · Ambiente: n8n Cloud `ederbox.app.n8n.cloud` (v2.87.0) e `gc-dev` (`gzbmhgnpoeormnidmgg`, só leitura) · Plano: `docs/automacao/migracao-telegram-integracao-automacao.md`, Fase 0 · Runbook: `docs/automacao/fase-0-runbook.md`

Resumo

Fase 0 concluída. Os seis itens de aceite do runbook estão fechados, a **decisão 6 está resolvida: o CHECK de `documentos_processamento` não muda**, e as sete camadas do `npm run validar` passam (seção 5.1).

A Parte A (Telegram, manual) foi feita pelo Breno, com um desvio: os bots foram apagados e recriados no meio, o que zerou o privacy mode e custou uma volta. A Parte B (n8n) está inteira. Nenhum token entrou em nó HTTP, documento, commit ou log — os dois vivem só dentro das Credentials.

Um achado fora do previsto, na seção 2: **uma premissa da decisão 6 estava factualmente errada**. A conclusão da decisão continua valendo; a premissa não.

1. O que existe agora

1.1 Os dois bots

	USERNAME	ID	CAN_READ_ALL_GROUP_MESSAGES
Teste	<code>@obraminds_ingestao_teste2_bot</code>	<code>8857741289</code>	<code>true</code>
Produção	<code>@obraminds_ingestao_v2_bot</code>	<code>8651400641</code>	<code>true</code>

Os usernames **não** são os sugeridos no runbook: os bots foram recriados no meio da Parte A, e os nomes originais não voltaram a ficar disponíveis. `can_read_all_group_messages: true` é o nome que a API do Telegram dá ao **privacy mode desligado** — foi assim que a decisão 5 foi verificada, e não pela tela do BotFather, que já tinha enganado uma vez (seção 4).

1.2 As duas Credentials

NOME	TIPO	ID
Telegram – ObraMinds Ingestão (teste)	telegramApi	wNL9xyZz7Z9lInBz
Telegram – ObraMinds Ingestão (produção)	telegramApi	wtb0bcZ5ituuVNek

Os tokens estão só aqui. Não há token em nó HTTP, em `.env.local`, neste documento nem em nenhum outro arquivo do repositório.

1.3 O grupo de admin

Grupo "Obras Automacao", `type: group` (grupo básico, não supergrupo), `chat_id` negativo de 10 dígitos. Guardado em **um lugar só**, como manda a decisão 5: `.env.local` → `TELEGRAM_ADMIN_CHAT_ID` (`.env.local` não é versionado).

O número não é reproduzido neste documento de propósito — a decisão 5 existe justamente para ele morar num lugar só, em vez de espalhado como o `558598202307` de hoje. Repetir aqui criaria o segundo lugar, que é o que diverge depois.

Registro do formato, que importa: por ser **grupo básico**, se ele virar supergrupo o `chat_id` muda para o formato `-100...` e o valor guardado para de funcionar. Está na tabela de armadilhas do passo a passo.

1.4 O workflow descartável

`KWqlgcMAYckvi0lJ` — **DESCARTÁVEL — Fase 0 · prova do Telegram Trigger (teste)**. Telegram Trigger (Credential de teste) → Code que expõe o `chat_id`, mais uma sticky note dizendo que é para apagar. Não grava nada e não chama nenhuma API.

Deixado inativo. Enquanto ativo, ele prende o único webhook que o token de teste admite, e isso quebraria o `getUpdates` de quem for depurar depois.

Apagado em 2026-09-22, a pedido do Breno, depois de servir de evidência da execução `3876`.

2. Decisão 6 — fechada, e o CHECK não muda

2.1 O que o fluxo de contrato grava

`d0t8aiCX20Cr08RH` (Integração Oficial — Contrato), nó `Supabase - Atualizar (APROVADO)`:

```
PATCH /rest/v1/documentos_processamento?id=eq.{{documentoId}}
{ status: "APROVADO", tipo_documento: "CONTRATO", dados_extraidos: {...} }
```

`APROVADO` **está** dentro do CHECK. Logo, pela decisão 6: a proposta usa `APROVADO` também, com `proposta_criada_id` preenchido, e **o CHECK fica como está**. Nenhuma condição de "pare e avise" disparou.

2.2 O CHECK, conferido ao vivo

Não lido do DDL — consultado em gc-dev em 2026-09-22, via `pg_constraint`. As 3 CHECK constraints da tabela:

```
documentos_processamento_status_check
    CHECK (status = ANY (ARRAY['PENDENTE', 'ERRO_VALIDACAO', 'REVISAO_HUMANA', 'APROVADO']))
documentos_processamento_canal_check
    CHECK (canal IS NULL OR canal = ANY (ARRAY['WHATSAPP', 'TELEGRAM']))
documentos_processamento_tipo_documento_check
    CHECK (tipo_documento = ANY (ARRAY['PROPOSTA', 'CONTRATO']))
```

2.3 Correção de uma premissa da decisão 6

A conclusão da decisão 6 está certa e confirmada. Uma premissa dela não está, e isso fica registrado em vez de corrigido no plano por conta própria.

A decisão 6 afirma que `PROPOSTA_REGISTRADA` não está no CHECK e que, por isso, *"o update do 4b falha hoje"*. Conferido no `I4wdVriPgS2W5MZP` (Integração Oficial — Proposta): o nó se **chama** `Supabase - Atualizar (PROPOSTA_REGISTRADA)`, mas o corpo que ele envia é

```
{ status: "APROVADO", tipo_documento: "PROPOSTA", dados_extraidos: {...} }
```

`PROPOSTA_REGISTRADA` só existe no rótulo do nó, nunca no payload. O update não falha — e não falhava. A premissa foi inferida do nome do nó, não do corpo.

O runbook pedia registrar "inclusive se o update vinha falhando com erro ou em silêncio". A resposta é: **nenhum dos dois**. Ele grava `APROVADO` e passa.

Distribuição em gc-dev, 2026-09-22 — **29 documentos**, o mesmo total da Fase 1:

TIPO_DOCUMENTO	STATUS	LINHAS	COM PROPOSTA_CRIADA_ID	COM CONTRATO_CRIADO_ID
CONTRATO	APROVADO	7	0	3
CONTRATO	PENDENTE	7	0	0
CONTRATO	REVISAO_HUMANA	10	0	0
PROPOSTA	APROVADO	1	0	0
PROPOSTA	ERRO_VALIDACAO	1	0	0

TIPO_DOCUMENTO	STATUS	LINHAS	COM PROPOSTA_CRIADA_ID	COM CONTRATO_CRIADO_ID
PROPOSTA	PENDENTE	2	0	0
PROPOSTA	REVISAO_HUMANA	1	0	0

A linha PROPOSTA · APROVADO · 1 é a prova de que o update do 4b chegou a rodar e passou.

Dois efeitos práticos, nenhum deles muda o que a decisão manda fazer:

1. **proposta_criada_id** é 0 nas 29 linhas. O 4a tem um segundo PATCH (Vincular Documento ao Contrato , que grava contrato_criado_id — daí as 3); **o 4b não tem equivalente**. Preencher isso é trabalho da Fase 6, e agora tem um espelho pronto para copiar.
2. **O 4b não tem nenhuma execução registrada** no n8n. O histórico do workflow está vazio.

3. Comportamento do Telegram Trigger em n8n Cloud — medido, não descrito

Item 3 do runbook. 5 leituras de `getWebhookInfo` no token de teste, uma por transição:

MOMENTO	URL REGISTRADA
Antes de tudo	"" (vazia)
Ao ativar o workflow	https://ederbox.app.n8n.cloud/webhook/610fa7c7-.../webhook
Ao desativar	"" — desregistra de verdade
Ao reativar	a mesma URL, mesmo UUID
Ao desativar no fim	""

O que isso estabelece, e que evita perder uma hora depois:

- **Ativar registra, desativar desregistra.** Não é preguiçoso nem adiado: o `getWebhookInfo` reflete na hora.
- **O UUID do caminho é estável** entre desativar e reativar. Ele é do nó, não do registro — então a URL de produção de um workflow não muda por ligar e desligar.
- **URL de teste e de produção são distintas.** A de produção é `/webhook/<uuid>/webhook`, e é a que a ativação registra. A de teste (`/webhook-test/...`) só vale enquanto o "Listen for test event" está aberto na tela — e, como o token admite **um** webhook só, abrir a escuta de teste **derruba o registro de produção daquele bot**. É exatamente o motivo de existirem dois bots (decisão do runbook, A1).
- `allowed_updates` ficou `["message", "callback_query"]`, embora o nó tenha sido configurado só com `message` — o n8n acrescenta `callback_query` por conta própria.

Estado final: os dois bots sem webhook registrado. O de produção nunca teve nenhum; o token dele só foi usado para `getMe`.

4. A Parte A, e o que ela custou

A Parte A é manual e foi do Breno. Três coisas deram errado, e as três viraram entrada na tabela de armadilhas do passo a passo, porque nenhuma é óbvia:

1. **401 Unauthorized** no `getUpdates`. Token errado na URL. Motivou um passo novo no guia — **Passo 4, conferir os dois tokens com `getMe`** antes de criar grupo e mexer em privacy, para o token não ser depurado junto com o resto.
2. **`/setprivacy` com "Success!" enganoso.** Digitar `disable` como texto, em vez de tocar no botão, faz o BotFather responder **Success! The new status is: ENABLED.** Começa com "Success", e passa por certo. O bot de produção ficou ligado assim.
3. **Bot recriado volta ao padrão `ENABLED`.** Os bots foram apagados e recriados no meio da fase; o privacy mode é por bot e não se herda. O sintoma foi preciso: no grupo só chegava mensagem que **mencionava** o bot, e o `oi` sumia.

Prova final de que a 3 foi resolvida: a mensagem `teste fase 0` foi mandada no grupo **sem menção** e chegou. Isso prova o privacy desligado *dentro daquele grupo*, que é diferente de estar desligado na configuração do bot.

5. Números reais

VERIFICAÇÃO	RESULTADO
Tokens validados por <code>getMe</code>	2/2 <code>ok:true</code> , usernames e ids conferidos
Privacy mode pela API	2/2 com <code>can_read_all_group_messages: true</code>
Credentials criadas	2 (<code>telegramApi</code>)
Workflows na instância	8 — 7 ativos + 1 inativo (SLA <code>TksH6ervKPA7Liar</code> , que a decisão 8 manda auditar e não reativar)
Workflows de produção alterados	0
Workflow descartável	1, 3 nós (2 executáveis + 1 sticky)
Validação do descartável	<code>valid: true</code> , 0 erros, 0 warnings
Execuções do descartável	1 , status <code>success</code> , 1763 ms , 1 item de saída

VERIFICAÇÃO	RESULTADO
<code>chat_id</code> capturado no log	<code>chat_tipo: "group"</code> , título <code>Obras Automacao</code> , texto <code>teste fase 0</code> , sem menção
<code>chat_id</code> pessoal	<code>884349214</code> , visto 2 vezes : como <code>from.id</code> no grupo e como <code>chat.id</code> do privado (<code>type: "private"</code> , via <code>getUpdates</code>)
Leituras de <code>getWebhookInfo</code>	5 no bot de teste + 1 final no de produção
CHECK constraints conferidas em gc-dev	3 na <code>documentos_processamento</code>
Linhas em <code>documentos_processamento</code>	29, distribuídas em 7 pares tipo/status
Linhas com <code>proposta_criada_id</code>	0 de 29
Itens de aceite do runbook	6/6

5.1 `npm run validar` — sete camadas, 2026-09-22, contra gc-dev

Rodado **depois** de conferir que a sessão da sprint 5 não tinha nenhum `next`, `tsc` ou `validar` em execução, nada escutando na porta 3000, e nenhum arquivo de `src/` ou `scripts/` alterado nos 10 minutos anteriores — as camadas build e navegador regravam o `.next`, e rodar por cima de outra sessão ativa a atrapalharia. `exit 0`.

CAMADA	BASELINE DA FASE 1 (21/09)	AGORA	
estático	<code>tsc</code> limpo, lint sem warnings	<code>tsc --noEmit</code> limpo, <code>✓ No ESLint warnings or errors</code> (6s)	ok
unitário	51 casos	106 casos, 106 pass, 0 falhas (1s)	ok
build	31 rotas	32 rotas, iguais à baseline (69s)	ok
runtime	41/41 rotas	44/44 rotas ok como <code>breno@obraminds.com</code> (25s)	ok
dados	12/12 checagens	15/15 checagens ok sob RLS (3s)	ok
escrita	38/38 passos	73/73 passos ok , limpeza dos 5 itens de teste, nada sobrou em gc-dev (24s)	ok
navegador	24/24 passos	41/41 passos ok , nenhum erro de console (96s)	ok

Nenhum desses números mudou por causa da Fase 0, e é isso que eles provam: a Fase 0 não tocou código da aplicação, e nada quebrou. Todo o crescimento em relação à baseline de 21/09 vem da **sprint 5 (Itens)**, que está no working tree da mesma branch — helpers de item, a aba Itens, a

importação e os testes correspondentes. Os números registrados aqui descrevem o estado do working tree em 22/09, com o trabalho da sprint 5 dentro, **não** uma entrega da Fase 0; quem fechar a sprint 5 os registra como dela.

Uma nota que o próprio plano imprime e não é regressão: `shell em 390px: 552px de scroll para 390px de janela` — pendência já conhecida do bloco 8.2.

Aceite do runbook, item por item

ITEM	
os dois bots respondem <code>/start</code>	ok
os dois no grupo, privacy desligado	ok — verificado pela API, não pela tela
as duas Credentials existem no n8n	ok
workflow descartável recebe mensagem e mostra o <code>chat_id</code> no log	ok — execução <code>3876</code>
<code>chat_id</code> do grupo anotado em um lugar só	ok — <code>.env.local</code>
decisão 6 fechada e registrada	ok — seção 2

6. O que não foi validado

Nominal, como manda o `CLAUDE.md`.

1. O `npm run validar` **não exercita nada da Fase 0**. As sete camadas passaram (5.1), mas elas cobrem a aplicação Next.js e o banco, e a Fase 0 vive no n8n e no Telegram. Nenhuma camada sabe que existem bots, Credentials ou webhook. O que provou a Fase 0 foram as verificações da seção 5 (`getMe`, `getWebhookInfo`, a execução `3876`), feitas à mão — **não há teste automatizado para nada disto**, e uma regressão nas Credentials ou no privacy mode não seria pega por `npm run validar`.
2. O **bot de produção nunca recebeu mensagem**. Só `getMe` e `getWebhookInfo`. O trigger foi exercitado **exclusivamente** com o bot de teste, que é o desenho. Que o de produção receba em grupo está inferido de `can_read_all_group_messages: true`, **não observado**.
3. O **envio do bot para o grupo não foi testado**. Nada foi mandado *para* o grupo por nenhum dos bots — o `chat_id` foi provado como **destino de leitura**, não de escrita. Quem primeiro vai escrever ali é o `Notificar`, na Fase 2.
4. **Conversa privada: vista com o bot de teste, pelo `getUpdates`; não passou pelo trigger**. O Breno mandou `/start` no privado do bot de teste e leu o `getUpdates`: `chat.type: "private"`, `chat.id = from.id = 884349214`, o mesmo número que tinha aparecido como remetente no grupo. Isso confirma o `chat_id` pessoal e confirma que chat privado e usuário têm o mesmo id. **Não foi exercitado**: a mensagem privada chegando pelo Telegram Trigger (o workflow estava desativado, e por isso o `getUpdates` funcionou), e qualquer conversa privada com o bot de **produção**. O caminho "cliente manda documento no privado" completo é da Fase 4.

5. `allowed_updates` não foi exercitado além de `message`. `callback_query` entrou sozinho no registro e nunca disparou.
 6. Nada foi escrito em gc-dev. As consultas da seção 2 são `select`. A Fase 0 não tem migration, não tem seed e não altera linha nenhuma.
 7. Conferência visual: não se aplica. A Fase 0 não tem tela.
-

7. Registro para quem for operar gc-prod

Nada da Fase 0 é tarefa de prod, e nada dela foi aplicado em prod. Fica o registro:

1. As Credentials são da instância n8n, não do banco. Como a instância `ederbox.app.n8n.cloud` é uma só, as duas Credentials já valem para qualquer ambiente — não há "recriar em prod".
 2. A decisão 6 não gera migration. O CHECK de `documentos_processamento` não muda, nem em dev nem em prod. Era o principal risco desta fase e ele não se materializou.
 3. O `TELEGRAM_ADMIN_CHAT_ID` vai precisar existir no ambiente de prod quando a Fase 2 subir — hoje ele só está no `.env.local` da máquina do Breno, que não é versionado.
 4. Segue valendo o da Fase 1: `20260921150000_contatos_canal.sql` e `criar_usuario_automacao.sql` não foram aplicados em prod, e o `supabase migration repair --status applied` das dez versões continua pendente lá.
-

8. Pendências que a Fase 0 deixa nomeadas

Atualizado em 2026-09-22, ao fim da Fase 2 — o que já foi resolvido está marcado.

1. ~~Apagar o `kw1gcMAYekvi013`~~ - Feito em 22/09, a pedido do Breno.
 2. ~~Corrigir a premissa da decisão 6 no plano (seção 2.3):~~ Feito: nota de correção na decisão 6 do plano, com a conclusão mantida.
 3. ~~proposta_criada_id nunca é preenchido~~ Resolvido em 22/09: o `Processar Documento` preenche ao aprovar proposta — provado com PDF real (ver `fase-2-notificar-e-simplificacao-status-entrega.md`, seção 4.2).
 4. Os usernames dos bots divergem do runbook (`teste2`, `v2`). Onde o plano ou o passo a passo citarem os nomes antigos, estão desatualizados.
 5. A chave secreta do Supabase está em texto claro nos headers dos nós HTTP dos workflows antigos. Pendente Resolvido em 22/09: os workflows antigos foram apagados, e o SLA, último com a chave escrita no nó, passou a usar Credential. Varredura dos 3 restantes: 0.
-

9. Próximo passo

Fase 2 — o sub-workflow `Notificar`, que é quem finalmente usa o `TELEGRAM_ADMIN_CHAT_ID` num nó só e substitui os ~12 hardcodes de `558598202307`.

A Fase 0 não criou o `Notificar`, não editou nenhum dos 7 workflows ativos e não criou a ingestão por Telegram — Fases 2, 3 e 4, nesta ordem.